Documentación Práctica 1

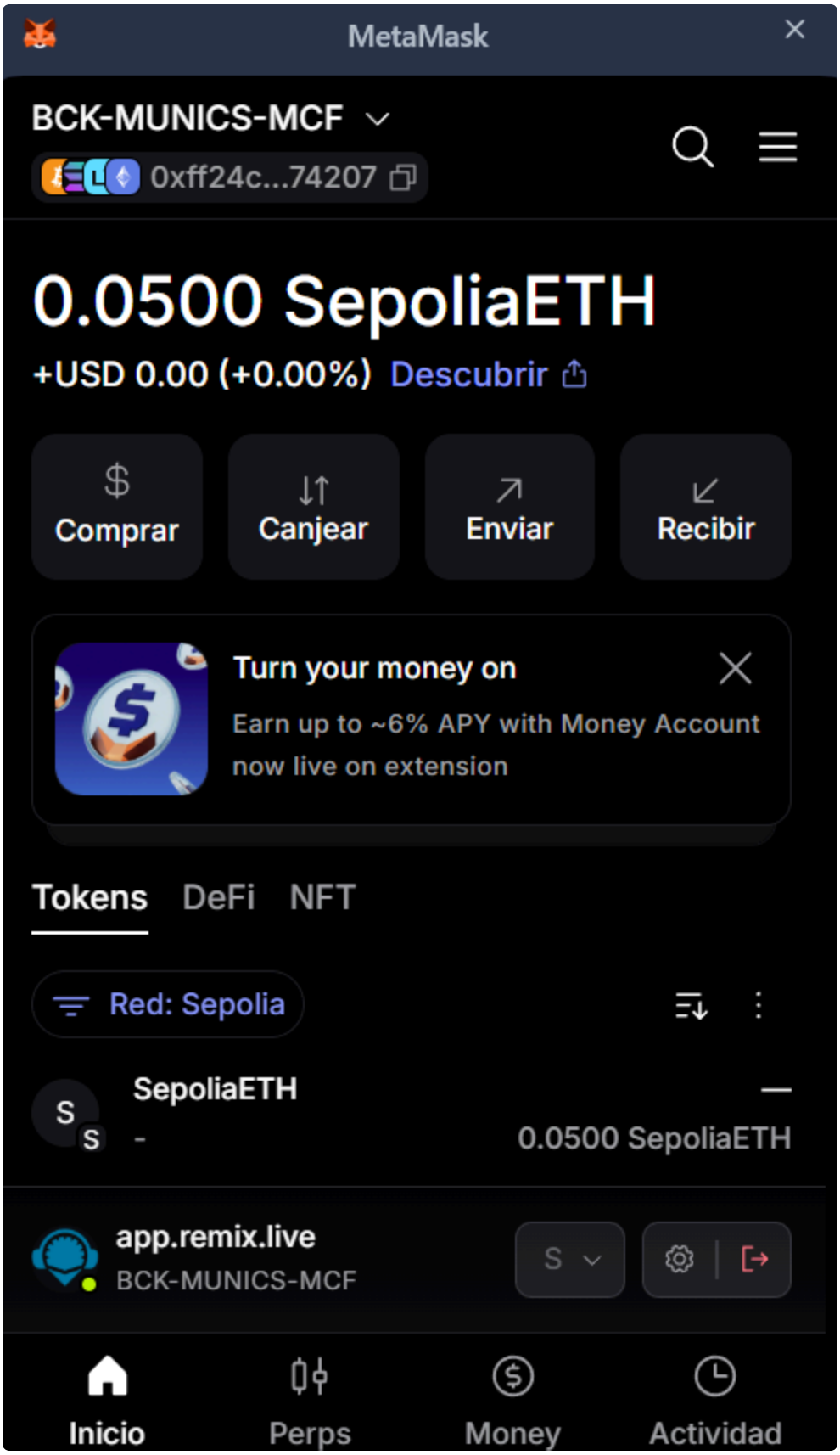
Ejercicios 1 y 2

Código implementado en el repositorio

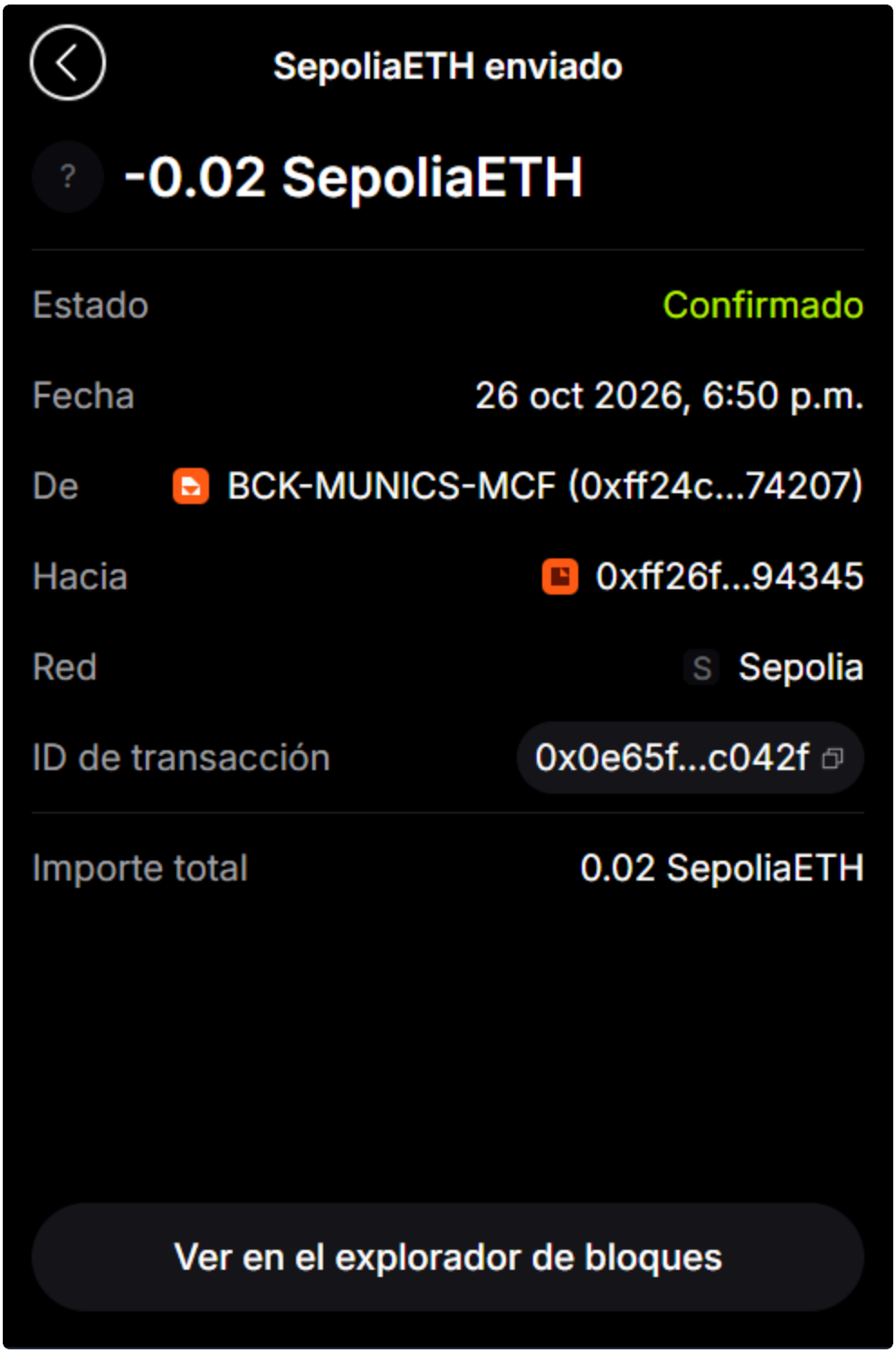
Ejercicio 3

Se obtuvo el Eth Sepolia a través del enlace de google cloud. Las siguientes imágenes demuestran la realización de la transferencia.

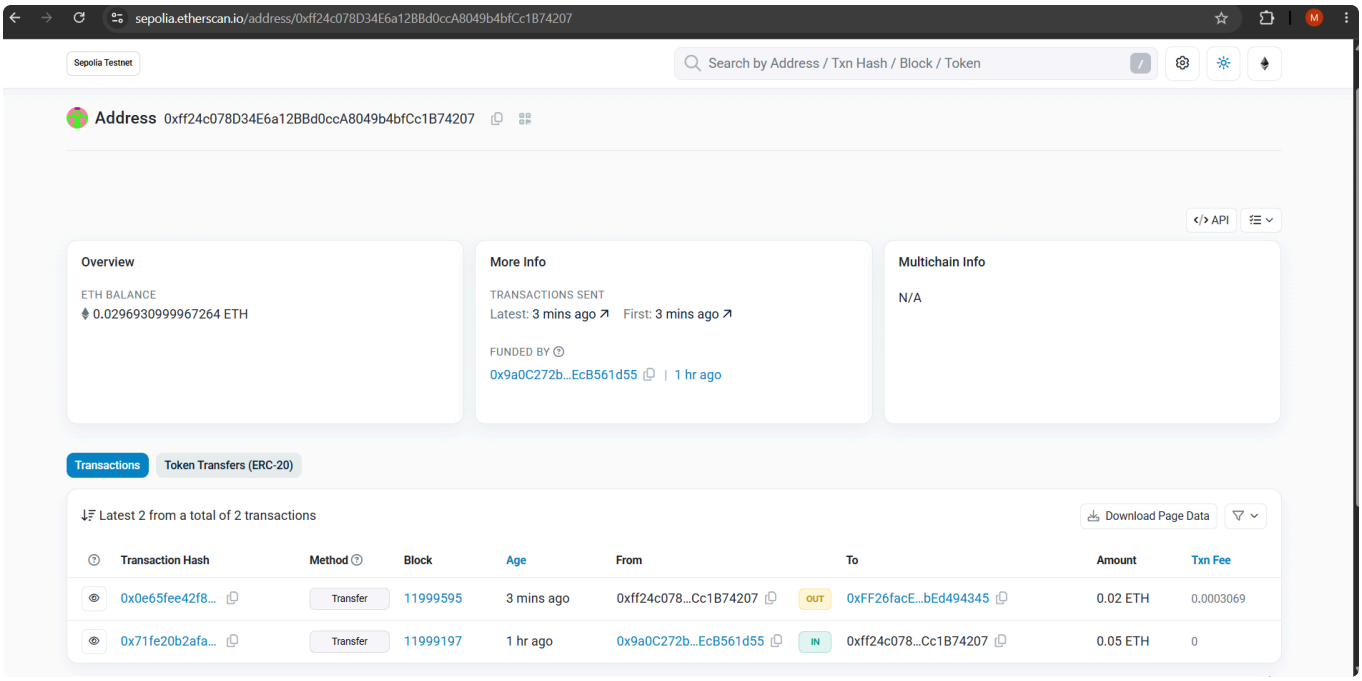
Cartera antes de la transacción:



Transacción realizada:



Transacción desde sepolia etherscan:



Ejercicio 4

El contrato implementado es un contrato de ejemplo de la libreria OpenZepelin que define funcionalidades de control de acceso para un Token ERC-20

Ejercicio 5

Análisis y definición del escenario

El contrato representa una plataforma de **Crowfunding por hitos**. Cada hito salvo el primero (los fondos iniciales para comenzar el proyecto), debe ser aprobado por los mecenas en una votación ponderada según lo aportado. De esta forma los *fondos quedan custodiados por el contrato y se van liberando por hitos*

Si la campaña no alcanza el objetivo, si un hito es rechazado o si el creador abandona el proyecto, los mecenas recuperan la parte proporcional de los fondos que todavía no se han liberado.

Diseño

El flujo de una campaña de Crowdfunding, que recoge todos los casos de uso del contrato es el siguiente:

1. El creador da de alta la campaña indicando: objetivo, duración, hitos e importe de cada hito.
2. Los mecenas aportan ETH, cuando se alcanza el primer hito los fondos se liberan automáticamente

3. Cuando se completa un hito, el creador abre una votación de 7 días en la que los mecenas decidirán si se ha alcanzado satisfactoriamente
4. Los mecenas votan
5. Si el hito se aprueba los fondos se liberan y el creador los puede retirar. En el caso de no aprobarse el hito, la campaña se cancela y cada mecenas recupera su parte de lo restante.

Contenido

- Datos

Estructura	Campos
Campaña	creador, fechaLimite, numHitos, hitoActual, estado, objetivo, recaudado, ultimaActividad, liberado
Hito	importe, votosFavor, votosContra, finVotacion
Mecenas	aportacion, votoEnHito, reembolsado
saldoCreador	direccion → wei pendientes de retirar

- Funciones

Función	Quién la llama	Estado requerido	Efecto
`crearCampana(objetivo, duracion, importesHitos)`	Creador	—	Alta de campaña y sus hitos
`aportar(id)` *payable*	Mecenas	Financiación	Suma la aportación. Si completa el objetivo, libera el hito 0
`solicitarHito(id)`	Creador	En curso	Abre una votación de 7 días
`votar(id, aFavor)`	Mecenas	Votación	Voto ponderado; aprueba o rechaza de forma anticipada si hay mayoría
`cerrarVotacion(id)`	Cualquiera	Votación, plazo vencido	Aplica quórum y mayoría
`declararAbandono(id)`	Cualquiera	En curso, 90 días sin actividad	Cancela la campaña

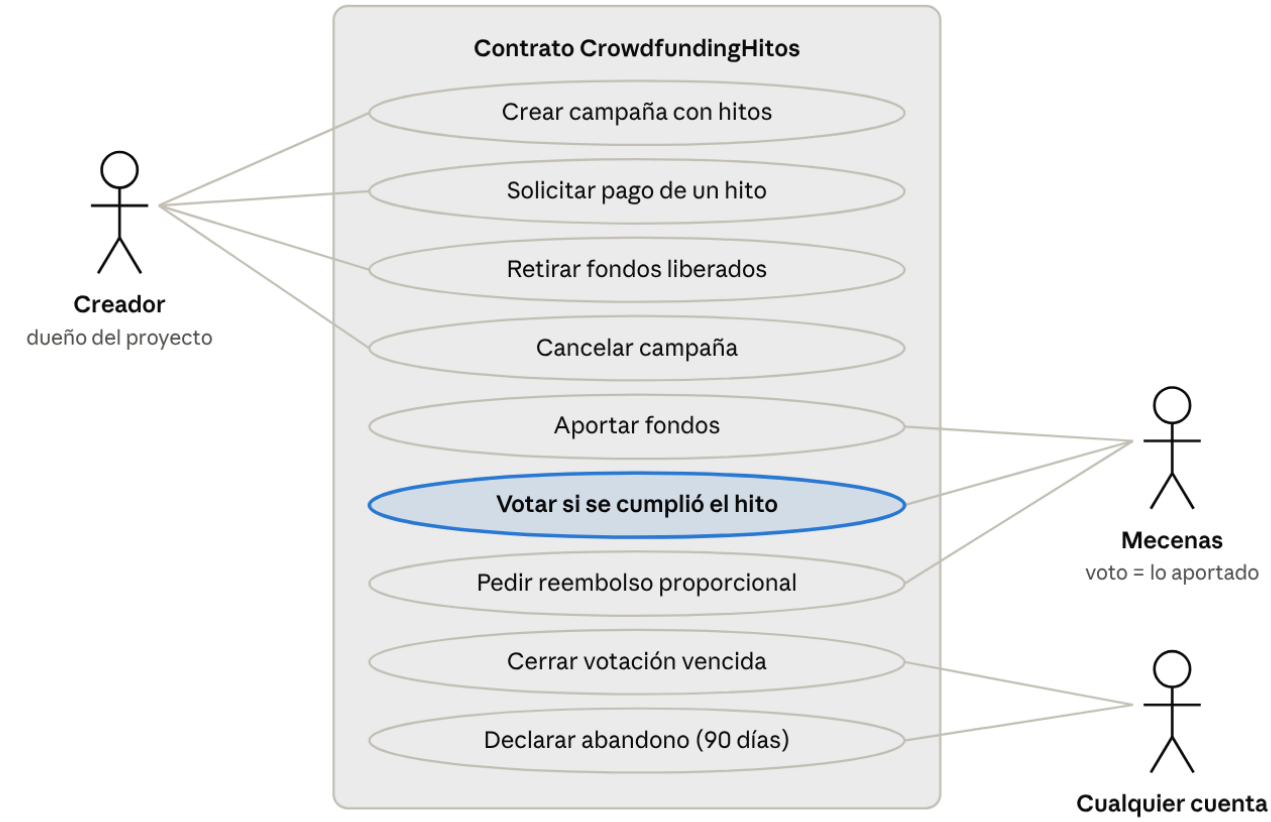
Función	Quién la llama	Estado requerido	Efecto
`cancelarCampana(id)`	Creador	Financiación o En curso	Cancela la campaña
`reembolsar(id)`	Mecenas	Fallida o Cancelada	Devuelve la parte proporcional no liberada
`retirar()`	Creador	—	Cobra los hitos liberados
`reembolsoDisponible(id, cuenta)` *view*	Cualquiera	—	Consulta cuánto recuperaría un mecenas

Estas funciones emiten eventos que permiten realizar una trazabilidad de las campañas

- Usuarios

En el diagrama aparece así por claridad, pero una cuenta de nuestro contrato puede desempeñar los 3 papeles. Las cuentas funcionan como un actor u otro en función de su relación con la campaña.

Tres tipos de usuario y nueve casos de uso



Implementación

La implementación se encuentra en el repositorio, dividida en los apartados de: Tipos de dato, Parámetros, Estado, Eventos, Errores, Creador, Mecenas, Usuario y funciones privadas.

Se utilizaron las siguientes **técnicas de optimización de gas**:

- Uso de `constants` para valores fijos, las constantes no ocupan storage y cuesta muy poco gas leerlas.

```
uint256 public constant DURACION_VOTACION = 7 days;
uint256 public constant PLAZO_ABANDONO = 90 days;
uint256 public constant DURACION_MAX_FINANCIACION = 90 days;
```

- Ajuste de tipo de variables. El tipo de variable apra cada variable se selecciona de forma que no reservemos espacio innecesario.

```
uint8 numHitos;
```

- Empaquetado de structs. El *storage* se organiza en *slots* de 32 bytes, que el compilador organiza por orden de declaración de las variables. Los structs se ordenan consecuentemente:

```
struct Campana {
        // slot 0: 20 + 5 + 1 + 1 + 1 = 28 bytes
        address creador;
        uint40 fechaLimite;     // fin del periodo de financiación
        uint8 numHitos;
        uint8 hitoActual;       // índice del siguiente hito pendiente de liberar
        Estado estado;
        // slot 1: 12 + 12 + 5 = 29 bytes
        uint96 objetivo;
        uint96 recaudado;
        uint40 ultimaActividad; // para detectar abandono del creador
        // slot 2 (12 bytes)
        uint96 liberado;        // total ya entregado al creador
    }
```

- Uso de punteros *storage* en vez de copiar variables.

```
Campana storage c = campanas[id];
```

- Uso de *unchecked* cuando la lógica de negocio protege a la operación de desbordamiento.

```
unchecked { favor = hito.votosFavor + peso; }
```

- Uso de *calldata* en parámetros de sólo lectura

```
function crearCampana(uint96 objetivo, uint40 duracion, uint96[] calldata importes
```

- Errores personalizados, el despliegue y las reversiones son más baratas que con un texto fijado.

```
error DatosInvalidos();
error NoAutorizado();
error EstadoInvalido(Estado actual);
error ExcedeObjetivo();
///=======================================
if (suma != objetivo) revert DatosInvalidos();
```

Pruebas

El fichero contiene las pruebas unitarias del contrato `CrowdfundingHitos` para el plugin *Solidity Unit Testing* de Remix. Antes de cada prueba se crea una campaña de 3000 wei dividida en tres hitos de 1000, con un creador y tres mecenas: Ana aporta 1500 (50 %), Beto 1000 (33 %) y Carla 500 (17 %).

Cada participante es un contrato `Participante`, para que tenga su propia dirección y su propio saldo. Remix no permite funciones con parámetros en los contratos de un fichero de pruebas, así que `Participante` no tiene funciones propias: solo reenvía al contrato la llamada que le pasa el test. Las pruebas que necesitan ETH lo reciben con la anotación `/// #value`.

Contrato `Participante`

Función	Descripción
`receive()`	Permite recibir ETH (reembolsos y retiradas)
`fallback()`	Recibe un destino y una llamada codificada, y la ejecuta con el ETH recibido

Contrato `CrowdfundingHitosTest`

Función	Tipo	Qué hace o comprueba
`beforeAll()`	Preparación	Despliega el contrato `CrowdfundingHitos` una sola vez
`beforeEach()`	Preparación	Crea creador y mecenas nuevos y una campaña de 3000 wei en 3 hitos
`testCampanaCreada()`	Prueba	La campaña tiene el creador, los 3 hitos, el objetivo y el estado Financiación correctos
`testHitosQueNoSumanElObjetivo()`	Prueba	Crear una campaña cuyos hitos no suman el objetivo revierte
`testFinanciarLiberaPrimerHito()`	Prueba (`#value: 3000`)	Al completar el objetivo pasa a En curso y libera el primer hito (1000)
`testNoSeSuperaElObjetivo()`	Prueba (`#value: 4000`)	No se puede aportar por encima del objetivo
`testCreadorNoPuedeAportar()`	Prueba (`#value: 100`)	El creador no puede ser mecenas de su propia campaña
`testAprobarHitoYRetirar()`	Prueba (`#value: 3000`)	Con mayoría a favor se aprueba el hito y el creador retira 2000
`testNoSePuedeVotarDosVeces()`	Prueba (`#value: 3000`)	Un mecenas solo puede votar una vez por hito
`testRechazarHitoYReembolsar()`	Prueba (`#value: 3000`)	Con el 50 % en contra se cancela; reembolsos de 1000, 666 y 333
`testFlujoCompleto()`	Prueba (`#value: 3000`)	Se aprueban todos los hitos, la campaña finaliza y el creador cobra 3000
`_hace(p, valor, llamada)`	Auxiliar	Hace que un participante llame al contrato; devuelve si la llamada tuvo éxito

Función	Tipo	Qué hace o comprueba
`_hitos(ultimo)`	Auxiliar	Construye el array de hitos: 1000, 1000 y el último importe indicado
`_financiar()`	Auxiliar	Ana, Beto y Carla aportan 1500, 1000 y 500
`_estado()`	Auxiliar	Devuelve el estado actual de la campaña como número

Ejecución de las pruebas:

